IEEESTEC – 19th Student Projects Conference, Niš, 2026.
https://doi.org/organizer will add number later
Original scientific paper

Fiducial: Build Any Robust Multidisciplinary Product with AI Agents

Aleksa Zdravković
12th grade high school student
ETŠ "Nikola Tesla" Niš
Niš, Serbia
aleksazdravkovic@proton.me
ORCID: 0009-0004-1535-1839

***Abstract*—This paper presents Fiducial, an open-source platform for building products that span several disciplines – a printed case, a circuit board, firmware and a web application – with AI coding agents. Every fact about the product is declared once in typed files, every artifact is derived from those files by one command, and a continuous-integration gate fails when an artifact no longer follows from its declaration. Hard problems such as part placement, routing and solid modelling are handed to established engines. A public example, a USB-C temperature and humidity stick, is carried from one declaration to a routed board, a checked case, firmware and a web page. When 20 realistic mistakes were injected into two products, the gates caught 15 of them before merge.**

***Keywords*—AI agents, single source of truth, hardware–software co-design, constraint solving, PCB automation, continuous integration**

I. Introduction

While building Outreach [1], an off-grid messaging system made of radios, gateways, a companion app and printed cases, the author ran into the same problem more than once. The radio firmware and the app disagreed on the layout of a Bluetooth message, and a printed case did not fit the board it was made for. Neither part was wrong on its own. Each was designed in its own tool, the facts they shared were copied by hand, and one copy changed while the other did not. The mistake showed up only when the parts were put together.

AI coding agents [2], [3] now write a large part of the code in a project. That makes each copy faster to write, but it does not remove the copies. An agent working in the firmware repository does not know that the board changed, and an agent working on the case does not know which connector the board uses. Writing disconnected copies faster only gives an inconsistent product sooner.

Fiducial [4] was made to solve this from the other side: each fact is declared once, and everything that needs it is derived from it. The case, the board, the firmware's pin map and the web page's types are all outputs of the same declaration, regenerated by one command and checked by one gate (Fig. 1). Agents, and the person working with them, edit the declarations and never the outputs.

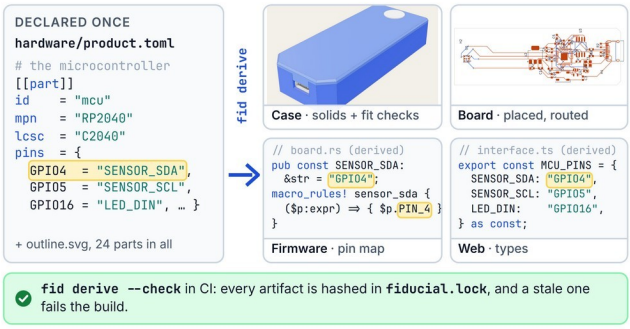


Fig. 1. One declaration, four disciplines. The sensor's data line is declared on GPIO4 once; fid derive places and routes the board, builds the case around it, and writes the firmware pin map and the web types from the same line.

II. System Overview and Implementation

Fiducial is a command-line tool, fid, written in Rust, together with 22 capabilities – packaged pipelines for hardware, firmware (RP2040 and STM32), web frameworks, translations, design systems, legal pages, deployment and more. A product turns on the capabilities it needs in its fiducial.toml file. The platform is organised in five layers, shown in Fig. 2.

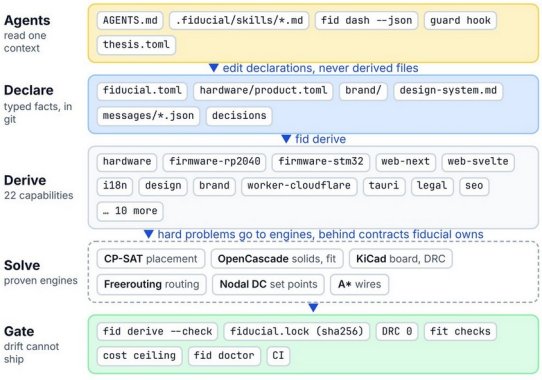


Fig. 2. The five layers of Fiducial. Agents read one shared context and edit declarations, capabilities derive the artifacts, engines solve the hard problems, and gates stop drift from being merged.

A. Declarations

A declaration is a typed text file kept in version control. Plain text was chosen over a modelling tool because it is what coding agents read and write best, and because every change then shows up in a normal code review. For hardware, the declaration is hardware/product.toml: the outline of the product, its case, and every part with an orderable LCSC number [5], a KiCad symbol and footprint [6], and the net each pin connects to. Placement intent is declared too – which zone a part belongs to, what it must be near or away from, and which part a vent seals over. Translations are declared once per language in messages/<locale>.json, and page metadata and colour contrast in the same way. A declaration only states facts. Nothing in it says where a resistor sits or how a trace runs.

B. Derivation and the Lock File

fid derive runs the pipeline of every enabled capability in order. Each one reads declarations and writes artifacts. The hardware pipeline alone writes the placement, the KiCad board and schematic, the bill of materials, the assembly order, a layout file that records why each position was chosen, the firmware pin map (board.rs) and the web types (interface.ts). The SHA-256 hash of every input and output is written to fiducial.lock, the same idea as a package manager's lock file or Nix's hashed builds [7]. Derivation is deterministic: run twice on the same inputs, it writes the same bytes. Without that, comparing hashes would mean nothing, so it was kept as a hard requirement throughout.

C. Engines Behind Contracts

The first version placed parts on the board with a hand-written search, and every new product requirement added another rule to it. The rules started to interact, and the code grew faster than the products. This was replaced: every hard problem now goes to an established engine, and Fiducial only owns the contract, the file that describes the problem. Part placement is a constraint model solved by CP-SAT from Google OR-Tools [8], and every placement rule is one of eight generic kinds (inside, no-overlap, fixed, flush, within, apart, near and net). The case is modelled in build123d on the OpenCascade kernel [9], which also checks that the board fits and the lid closes. KiCad lays out the board and runs its design-rule check (DRC), and Freerouting [10] routes it. A small nodal solver checks resistive set points against declared windows, such as the CC voltages of USB Type-C [11].

When two declarations contradict each other, the placement model has no solution, and the solver reports the smallest set of declarations that cannot all hold, written in the product's own terms (Fig. 3, case 3). The person then decides which one to relax.

D. Gates

fid derive --check recomputes the hashes and fails if an input changed since its outputs were derived, or if an output was edited by hand. It runs in continuous integration on every commit, so a stale artifact cannot be merged. Derivation also checks the product itself: the case must fit the board, the board must pass DRC with zero violations, the set points must be inside their windows, every language must have every string, and the parts must cost less than a declared ceiling. Finally, the firmware compiler and the web type checker read derived files, so renaming a fact breaks every place that still uses the old name. Fig. 3 shows three real failures from the example product.

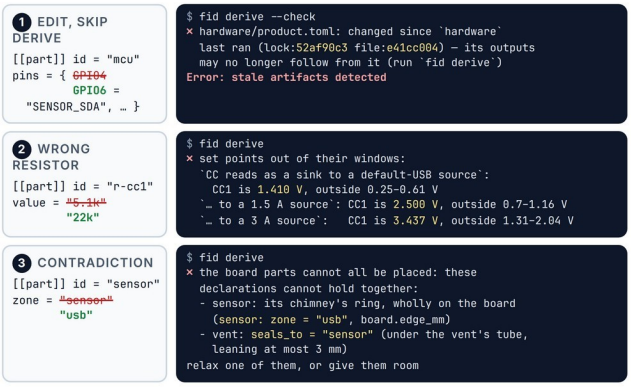


Fig. 3. Three gate failures on the example product: an edit whose outputs were not re-derived, a 22 kΩ resistor in place of 5.1 kΩ, and two placement declarations that contradict each other.

E. Working with AI Agents

Every product has an AGENTS.md file [12] and one instruction file per capability, so any agent – Claude Code, Codex, Cursor or others – starts from the same context. fid dash --json gives an agent the current state of the product, and a hook that runs before every shell command stops an agent from editing a derived file. The rule given to agents is that a mistake is fixed in the layer that owns the fact: a wrong pin in the declaration, a wrong derivation in the platform, and never in the output. Fiducial itself was built this way.

III. Worked Example: Sensor Stick

The first product Fiducial was used on is private, so a public example was built in the repository to show the whole chain on something anyone can inspect: a USB-C stick that reports room temperature and humidity to a web page (Fig. 4). It was kept small and cheap on purpose, but it still has every discipline: an RP2040 microcontroller [13] with its crystal and QSPI flash, an AHT20 sensor [14], a WS2812B status LED behind a 5 V buffer, a 3.3 V regulator and a USB-C socket – 23 parts on the board, USD 4.74 per board at five boards.

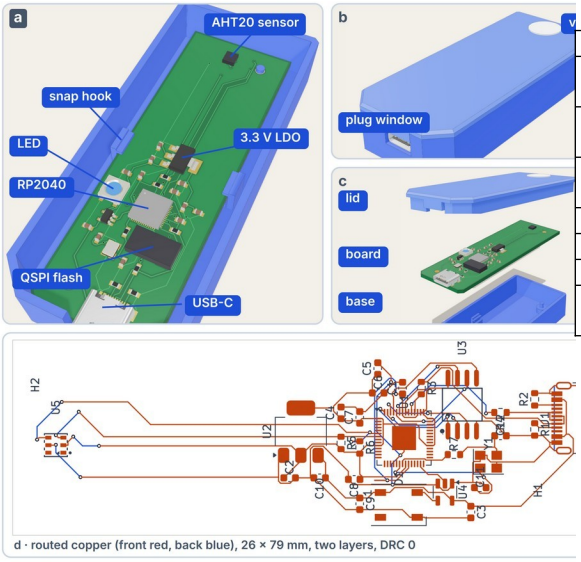


Fig. 4. The sensor stick, derived from one declaration: (a) the routed board in its base, with the sensor alone at the far end, away from the regulator's heat; (b) the closed case, 36 × 84 × 15.5 mm; (c) exploded view; (d) the routed copper.

The person writes product.toml, an outline drawing, the firmware and the page, and Fiducial derives the rest. The board is sized from the area of its parts (26 × 79 mm, two layers), placed by CP-SAT from 80 constraint instances of six kinds, routed, and passes DRC. The case is built around the board, with a window where the socket meets the wall and a vent sealed over the sensor. The sensor sits at the far end, away from the regulator, because a humidity sensor next to a warm part measures the part and not the room.

The firmware is written in Rust on Embassy [15] and takes every pin through the derived board.rs. Pins are given as macros rather than numbers, because in Embassy every pin is its own type: if a pin is moved to one the chip's I²C cannot use, the firmware stops compiling instead of failing on the bench. The page reads the sensor over the Web Serial API [16] and imports the derived interface.ts. A fabrication pipeline writes the Gerber files, the assembly bill of materials and the placement file a board house needs.

IV. Results

A. Measurements

Fiducial was developed over four weeks (repository created on 6 September 2026, release 0.9.4 on 4 October 2026) by one person working with AI agents: 195 commits on the main branch through 55 merged pull requests, about 60,000 lines of Rust, 12,000 of TypeScript and 6,000 of Python. It was used on two products, the public sensor stick and a private one whose design is not shown. Table I summarises the measurements.

TABLE I. Measured Results

Measure	Result
Placement rule kinds	8; none added for either product after the switch to the solver
Constraint instances	80 of 6 kinds (sensor stick); 78 of 7 kinds (private product)
Connection length (private)	−13 % vs. hand-written search; a further −10 % with a warm start
Board, sensor stick	26 × 79 mm, 2 layers, 0 DRC violations
Parts cost	USD 4.74 per board at 5; ceiling USD 15
One-line pin move	8 derived files rewritten in 31 s
Contradictions	reported as a minimal set of named declarations

To measure how a change spreads, the sensor's data line was moved from GPIO4 to GPIO8, one line in the declaration. Before deriving, --check failed the build (Fig. 3, case 1). A single fid derive then rewrote eight derived files in 31 s on a cloud container, among them the board, the schematic, the firmware pin map and the web types. The firmware and the page needed no edit.

Two findings changed the platform during this work. CP-SAT run with several worker threads gave different answers on the same input, even with deterministic time limits. Since every artifact is checked by hash, placement now runs on one worker with large-neighbourhood search, which repeats exactly and, starting from the previous answer, also found shorter connections. Freerouting is not deterministic either, so the routed board is checked (DRC passes and the nets match the declaration) rather than compared byte for byte, and a product can keep a hand-routed board that the build checks against its declaration.

B. Injected Mistakes

The measurements show that derivation works when it is used correctly. To see what happens when a person or an agent makes the mistakes that cause drift, 20 realistic mistakes were injected into two products, one at a time, and the gates were run in the same order as in CI. The products were the sensor stick and a fresh test product in English and Serbian with ten software capabilities turned on. Three correct changes and two no-change controls checked that the gates do not fail without a reason. Everything ran on release 0.9.4. Table II lists the mistakes.

TABLE II. Injected Mistakes and Where They Were Caught

Area	Mistake	Caught by
i18n	String added in English only	derive: missing in sr
i18n	English string edited, not re-derived	not caught
i18n	Derived message module edited by hand	--check
content	Post published in English only	derive: missing in sr
SEO	Page without title or description	derive: no metadata

brand	Domain changed, not re-derived	not caught
deploy	Derived Worker config edited by hand	--check
design	Text below 4.5 : 1 contrast	derive: palette check
database	Applied migration edited	--check
i18n	Serbian string left in English	not caught
hardware	Sensor pin moved, not re-derived	--check
firmware	Derived pin map edited by hand	--check, compiler
circuit	USB-C pull-down 5.1 kΩ → 22 kΩ	derive: set points
placement	Sensor zone contradicts its vent	derive: conflict
cost	Ceiling lowered to USD 3	derive: cost ceiling
interface	Net renamed, code not updated	compiler, type check
firmware	Pin used as a number, then moved	not caught
protocol	Reading fields swapped in firmware	hand-written test
firmware	Sensor I²C address mistyped	not caught
mechanical	Case narrowed 36 → 28 mm	derive: does not fit

Fifteen of the twenty were caught before merge: eight by checks during derivation, five by --check, and two by the compiler, the type checker or a test. Every message named the file and the declaration at fault, as in Fig. 3. Every gate answered in under a second, except the placement contradiction, which took 15 s because the solver has to prove that no placement exists. The correct changes passed every gate and the controls changed nothing.

The five misses are as useful as the catches. Two of them, a string and a domain edited without re-deriving, came from a gap in Fiducial itself: the software pipelines do not list the files they read, so --check compared the outputs with the lock but did not notice the changed input. The hardware pipeline lists its inputs, which is why the same mistake there was caught. This gap was found by the test and is still open. Another miss was firmware that uses a pin number directly instead of board.rs: it compiles before and after the pin moves, because derivation only protects the code that uses it. The last two were facts that were never declared or that need judgment: a mistyped sensor address exists only in the firmware, and a Serbian string that is really English passes every structural check. The swapped reading fields were caught only by a hand-written test, because the message layout is still written twice, in the firmware and in the page – the same mistake that started this work in Outreach.

V. Discussion

Better models move the line between what a person has to check and what can be left to the agent, but they do not remove it. A probabilistic system cannot be guaranteed to be correct, or to match what the product needs, and even a deterministic tool is only right about what it was told. Fiducial is the layer that keeps that randomness bounded: the declarations are the constraints, derivation is repeatable, and the gates refuse drift. During this work the gates caught every disagreement between artifacts, and the person still had to judge what no gate could: a parts catalogue that changed its format overnight, a text match that read “±0.1 %” as “1 %”, a sensor that had to be moved away from the board’s heat, and a case that its board did not fit until the board was cut to the case.

The approach is built around the product rather than around one codebase. Separate repositories, each with an agent that knows only its own part, give separate designs. One declaration read by every agent gives one product.

The evidence has clear limits. Only two products were built, one of them public. The injected mistakes were chosen by the author, who also wrote the platform. Set points are checked at DC only, and nothing is simulated in time. Autorouting is not deterministic. Most importantly, neither product has been manufactured and tested yet: everything here is designed and checked, not built.

VI. Conclusion

Fiducial shows that declaring each fact once, deriving every artifact from it and gating the result lets one person working with AI agents keep a case, a board, firmware and a web application in agreement. Handing hard problems to proven engines kept the platform small, since new requirements became new declarations instead of new rules. Of 20 injected mistakes, 15 were caught before merge, and the five that were not point to the next steps: track the inputs of every pipeline, declare message layouts once for firmware and web, and stop code from using pin numbers directly. Next, the sensor stick will be manufactured and tested on a bench, and the approach will be tried on products built by people other than the author. The platform and the example are open source [4].

VII. References

[1] A. Zdravković, Đ. Jovanović and P. Šubarević, “Outreach: Modular AI-enhanced off-grid mesh communication system for remote environments,” IEEESTEC – 18th Student Projects Conference, Niš, 2025.

[2] Claude Code documentation. Available at: https://docs.anthropic.com/en/docs/claude-code

[3] OpenAI Codex. Available at: https://openai.com/codex/

[4] A. Zdravković, Fiducial, release 0.9.4, 2026. Available at: https://github.com/AleksaZCodes/fiducial/releases/tag/v0.9.4

[5] LCSC Electronics component catalogue. Available at: https://www.lcsc.com/

[6] KiCad EDA. Available at: https://www.kicad.org/

[7] E. Dolstra, M. de Jonge and E. Visser, “Nix: A safe and policy-free system for software deployment,” in Proc. LISA, 2004, pp. 79–92.

[8] L. Perron and F. Didier, CP-SAT, Google OR-Tools. Available at: https://developers.google.com/optimization/cp/cp_solver

[9] build123d, a Python CAD library on the OpenCascade kernel. Available at: https://github.com/gumyr/build123d

[10] Freerouting, an open-source PCB autorouter. Available at: https://github.com/freerouting/freerouting

[11] USB Type-C Cable and Connector Specification, USB Implementers Forum. Available at: https://www.usb.org/usb-type-cr-cable-and-connector-specification

[12] AGENTS.md, a format for guiding coding agents. Available at: https://agents.md/

[13] RP2040 datasheet, Raspberry Pi Ltd. Available at: https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf

[14] AHT20 humidity and temperature sensor datasheet, Aosong. Available at: https://asairsensors.com/wp-content/uploads/2021/09/Data-Sheet-AHT20-Humidity-and-Temperature-Sensor-ASAIR-V1.0.03.pdf

[15] Embassy, an async framework for embedded Rust. Available at: https://embassy.dev/

[16] Web Serial API reference. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Web_Serial_API